A Shared Conversation Governor

for OpenClaw, OmegaClaw, and OmegaHive

Reducing token use and conversational noise without reducing useful capability

---

Architecture, detailed design, rollout strategy, and implementation plan

Prepared for Ben Goertzel

July 24, 2026

Scope. This document proposes a shared communication control plane for the current two-agent proto-hive, consisting of an OpenClaw agent with extensive skills (Protocosmobot) and an OmegaClaw agent that uses the OpenClaw gateway. Both agents currently use smart-ish model routers. The design extends this setup with event admission, responder arbitration, deduplication, private bot-to-bot review, update aggregation, and output governance. It also describes how the same architecture can scale to a richer OmegaHive spanning many agents and both AI-only and mixed human/AI channels.

Contents

1 Executive Summary

The current proto-hive contains two powerful but partially overlapping conversational actors:

- **Protocosmobot**, an OpenClaw agent with broad operational skills, tools, project access, cron workers, and the ability to inspect and modify external state.
- **OmegaClaw**, a more reflective or synthetic agent that uses the OpenClaw gateway and can provide deeper conceptual analysis, criticism, and long-form reasoning.
- Each agent has a smart-ish router that chooses which language model should answer a given message.

This arrangement has substantial functional value, but its present communication dynamics allow avoidable token use and noise. The major waste is not simply that useful answers are occasionally too long. A larger problem is that the system often creates messages that should never have been generated: acknowledgments of acknowledgments, reactions to status messages, narration of tool use, repeated error traces, duplicate explanations, visible `NO_REPLY` messages, and multiple public answers to one event.

The proposed solution is a shared OmegaHive Conversation Governor (OCG) placed at the common gateway boundary, before model selection and after model generation. The governor adds two control surfaces:

1. An **ingress and admission governor** decides whether any agent or language model should be invoked, which agent owns the public response, whether a private reviewer is useful, and what output contract applies.
2. An **egress governor** suppresses no-op output, rejects duplicates, assembles multipart responses, enforces response contracts, and prevents operational chatter from becoming public conversation.

The smart router is correspondingly expanded. Instead of choosing only among language models, it chooses among actions:

```
DROP | LOG_ONLY | TEMPLATE | CHEAP_MODEL | NORMAL_MODEL | DEEP_MODEL
```

The most important consequence is that many events terminate before any LLM call. A receipt, duplicate system event, repeated cron status, sibling echo, or non-addressed bot message can be recorded without invoking a model or posting a reply.

For the current two-agent system, the default responsibility split is:

- Protocosmobot is the **execution and evidence agent**. Its tools and workers emit structured facts, artifacts, blockers, approvals, and state changes.
- OmegaClaw is the **synthesis and reasoning agent**. It receives compact state plus selected evidence rather than raw operational streams.
- The governor grants exactly one **public response lease** per event or task state. A second agent may act as a private reviewer, but does not independently answer the human unless explicitly invited.

The proposed architecture preserves utility because it does not force all reasoning to become short or shallow. Deep analysis remains available when requested or justified. The design instead removes redundant invocations, moves operational detail into durable artifacts and logs, makes silence an actual control action, and uses progressive disclosure for optional detail.

For a larger OmegaHive, the same principles generalize into a multi-channel communication fabric with:

- typed channels and privacy labels;
- topic- and role-based subscriptions;
- local channel governors coordinated by a global event ledger;
- internal AI-only deliberation rooms;
- mixed human/AI rooms with one or more designated public spokesagents;
- bounded review committees rather than uncontrolled many-agent reply cascades;
- attention budgets, response leases, incident aggregation, and durable handoff artifacts.

The implementation can be staged safely. The first deployment should run in shadow mode, recording what would have been suppressed without actually suppressing anything. Once false-silence rates are acceptably low, deterministic no-op suppression and repeated-event coalescing can be activated, followed by shared response leases, private review requests, daily digest aggregation, and eventually multi-channel OmegaHive federation.

2 Motivation and Empirical Problem

2.1 The key distinction: verbosity versus message creation

There are two separable sources of token waste:

1. **Essay amplification**: a useful response is longer than necessary for the immediate decision.
2. **Message amplification**: a response is generated even though no response was needed, or several agents respond to the same event.

Length limits and concise prompting primarily address essay amplification. They are useful, but they do not address the larger structural failure modes. A bot that generates `NO_REPLY`, "noted," or "nothing to add" has already consumed model tokens. If this text is then posted into a shared channel, it can provoke another agent to spend tokens interpreting and acknowledging it.

The central design principle is therefore:

> **The cheapest and least noisy response is not a shorter LLM response. It is a correctly suppressed LLM invocation and no outbound message.**

2.2 Observed failure classes in the current proto-hive

The transcripts exhibit recurring classes of behavior that are individually understandable but collectively expensive:

Acknowledgment loops
: One agent says "noted," another acknowledges the note, and the first confirms that the acknowledgment is already covered.

Visible silence markers
A bot outputs `NO_REPLY`, "staying quiet," or "nothing to add" into the channel instead of returning an internal no-send status.

Parallel public answers
Protocosmobot and OmegaClaw each answer the same human message, often with overlapping substance.

Tool narration Internal steps such as "let me inspect," "continuing provisioning," or "now I will retry" are posted as public conversational messages.

System-event echoing Full stack traces, fallback notices, or cron failures are repeatedly injected into a channel and repeatedly analyzed.

State reconstruction A stateless worker rereads broad context and re-explains project history rather than consuming a compact current-state object.

Channel and identity confusion
An agent responds to a message meant for another agent, answers a question from a different project, or treats system metadata as a request.

Promise loops A bot repeatedly announces that it is reading or preparing a result but does not deliver a concrete artifact.

Speculative prose before cheap verification
Several paragraphs are produced about an operational uncertainty that could have been resolved by a simple file, process, commit, or job-status check.

Over-reporting incremental progress
Small test-count increases, routine commits, or "no material change" updates are sent many times per day.

These behaviors do not imply poor underlying reasoning. They arise because the current architecture gives each agent an incomplete local view and asks each model invocation to decide both *whether* to respond and *what* to say. The result is a conversational tragedy of the commons: locally reasonable agents jointly produce global noise.

2.3 Why prompt-only fixes are insufficient

Both agents should receive stronger communication instructions, but prompts alone cannot reliably enforce system-wide invariants:

- An agent cannot know that a sibling already claimed a message unless there is shared state.
- A model cannot save the tokens used to decide that it should say `NO_REPLY`; only pre-inference admission can do that.
- Retry and reconnect events may cause the same input to be processed more than once unless idempotency is enforced outside the model.
- A model may correctly identify an echo loop and still produce another message explaining the echo loop.
- Independent routers may both choose a sensible model for the same event; they need a shared arbitration layer, not merely better model selection.

The proposed governor therefore treats communication discipline as a first-class control-plane problem.

3 Goals, Non-Goals, and Design Principles

3.1 Primary goals

The system should:

1. Reduce total model input and output tokens without materially reducing useful functionality.
2. Prevent duplicate or no-op public messages before they are generated.
3. Preserve the value of having both an operational agent and a reflective reasoning agent.
4. Ensure one clear public answer by default, even when multiple agents contribute internally.
5. Aggregate routine project progress into compact, meaningful updates.
6. Keep detailed evidence and reasoning in durable artifacts rather than repeatedly regenerating it in chat.
7. Improve traceability: every public response should have a clear triggering event, owner, state version, and rationale.
8. Scale from the present two-agent proto-hive to a larger OmegaHive with many channels and agents.

3.2 Non-goals

The design is not intended to:

- force all answers into a uniformly short style;
- prevent rich philosophical, scientific, or technical dialogue when that dialogue is the point;
- replace agent-specific personalities, skills, or model routers;
- centralize all cognition in one monolithic controller;
- hide consequential disagreement among agents;
- prevent humans from explicitly requesting multiple independent answers;
- make the system silent merely to optimize cost.

3.3 Core design principles

1. **Admission before inference.** Decide whether an LLM call is needed before selecting a model.
2. **One public owner, many private contributors.** Multiple agents may assist, but one agent normally holds the public response lease.
3. **Silence is a real action.** No-send is structured control output, never user-visible prose.
4. **Delta over narrative.** Communicate changed state, decisions, blockers, and evidence; do not reconstruct unchanged history.

5. **Probe before prose.** Resolve cheap operational uncertainties before writing extensive analysis.
6. **Artifacts carry depth.** Chat carries summaries, decisions, and pointers; durable files carry long reasoning and evidence.
7. **Stateful infrastructure, stateless models.** Models may remain replaceable and session-local while the communication fabric maintains durable event, task, and project state.
8. **Fail open for usefulness only where safe.** Early deployment should prefer logging and shadow decisions over aggressive suppression; suppression becomes stricter only after measurement.
9. **Human intent is authoritative.** Explicit user addressing, request for multiple viewpoints, or request for live updates overrides default quietness policies.

4 Current-System Assumptions

The proposed design assumes the following approximate topology:

```
Telegram and other channels
        |
        +--> Protocosmobot / OpenClaw
        |       - skills and tools
        |       - project workers and cron jobs
        |       - smart-ish model router
        |
        +--> OmegaClaw
                - reflective/synthetic reasoning
                - uses the OpenClaw gateway
                - smart-ish model router
```

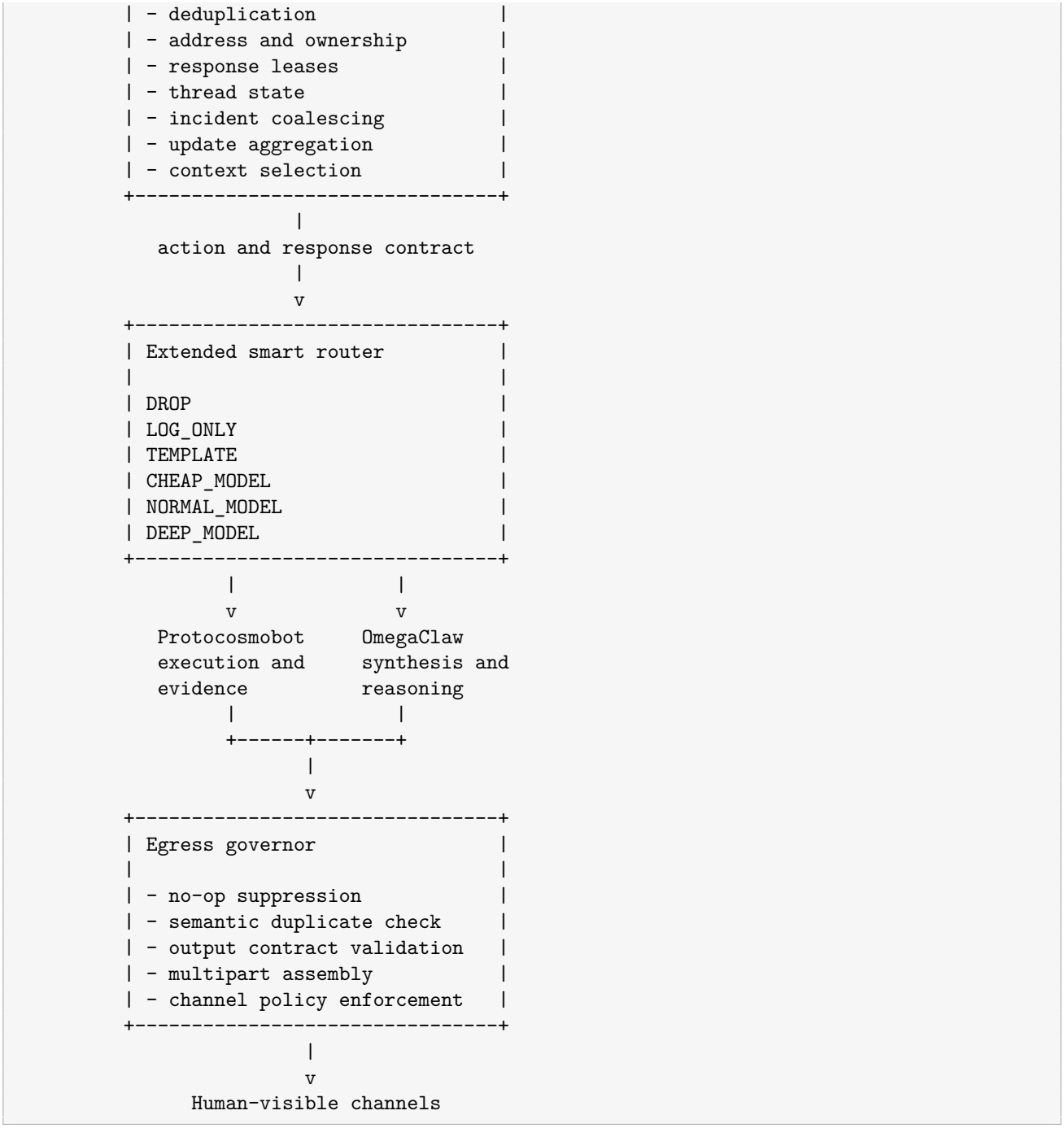
The critical architectural opportunity is that both agents ultimately interact through the OpenClaw gateway or a nearby shared transport layer. The governor should be placed at the earliest common point where inbound events can be inspected before either agent constructs a large prompt or invokes a language model.

If the current gateway is only a low-level inference proxy, the governor should be implemented immediately above it as a shared conversation service. If the gateway already receives channel events, schedules agent turns, and dispatches model calls, the governor can be integrated directly into it.

5 High-Level Architecture

5.1 Proposed topology

```
External channels, cron jobs, tools, project workers
                          |
                          v
              Event normalization layer
                          |
                          v
          +---------------------------------+
          | OmegaHive Conversation          |
          | Governor: ingress/admission     |
          |                                 |
```



5.2 Control plane versus cognition plane

The governor belongs to the **control plane**. It does not need to solve the user task. Its job is to decide:

- whether the event should cause any inference;
- which agent is responsible;
- whether a private review is warranted;
- how much context is needed;

- what cost and length budget applies;
- whether the output may be sent.

Protocosmobot and OmegaClaw remain in the **cognition and action plane**. They perform tools, reason, synthesize, and create artifacts. This separation is important because communication admission should be deterministic where possible, cheap, auditable, and shared across agents.

6 Detailed Component Design

6.1 Event normalization and provenance envelope

Every inbound item should be converted into a canonical event envelope. Sources include human messages, bot messages, tool results, cron completions, system errors, file arrivals, project-state changes, and internal review requests.

A proposed schema follows.

```
{
  "event_id": "evt_01J...",
  "event_type": "human_message",
  "timestamp": "2026-07-24T18:42:00Z",
  "source": {
    "channel_id": "telegram:protobots",
    "channel_type": "mixed_human_ai",
    "sender_id": "human:ben",
    "sender_kind": "human",
    "message_id": "tg:299803"
  },
  "addressing": {
    "explicit_agents": ["protocosmo"],
    "reply_to_agent": null,
    "reply_to_event_id": "evt_01H..."
  },
  "causality": {
    "caused_by_event_id": null,
    "origin_chain": ["human:ben"],
    "retry_of_event_id": null
  },
  "scope": {
    "project_id": "omegasim",
    "task_id": "a6-resweep",
    "thread_id": "thread:omegasim:a6",
    "state_version": 42
  },
  "content": {
    "text": "...",
    "attachments": [],
    "content_hash": "sha256:...",
    "semantic_fingerprint": "emb:v1:..."
  },
  "policy": {
    "privacy": "mixed-room",
    "urgency": "normal",
    "human_response_expected": true
```

```
  }
}
```

The envelope is critical for preventing loops. When a bot's own outbound message returns through Telegram, its origin chain remains visible. A sibling agent should not treat it as a fresh human request unless it is explicitly addressed or assigned for review.

6.2 Event admission

Admission is the first decision point. It should be implemented as a sequence of cheap checks, with an optional small classifier only after deterministic checks fail.

6.2.1 Deterministic drop conditions

An event can normally be dropped before inference when:

- its `event_id` or retry lineage was already handled;
- it is the agent's own outbound message returning through the channel;
- it is a sibling bot message not addressed to the current agent and no review assignment exists;
- it is a repeated system incident with no changed state;
- it contains only a visible silence marker or acknowledgment;
- the project state version is unchanged and the event is a routine update;
- a public response lease for the task is already held by another agent;
- it is an intermediate tool-progress event that channel policy marks as internal.

6.2.2 Admission decision object

```
{
  "event_id": "evt_01J...",
  "decision": "ADMIT",
  "reason_codes": ["EXPLICITLY_ADDRESSED", "HUMAN_QUESTION"],
  "public_owner": "protocosmo",
  "private_reviewers": ["omegaclaw"],
  "router_action": "NORMAL_MODEL",
  "response_contract_id": "direct_technical_answer_v2",
  "context_profile": "project_compact_plus_evidence",
  "lease_id": "lease_01J..."
}
```

Other decisions include `DROP`, `LOG_ONLY`, `AGGREGATE`, `TEMPLATE`, and `DEFER_UNTIL_COMPLETE`.

6.3 Addressing, ownership, and response leases

6.3.1 Default ownership rules

The governor should resolve ownership in this order:

1. Explicitly mentioned agent.
2. Agent whose message is being replied to.
3. Assigned project or task owner.
4. Channel-specific default spokesperson.
5. Role-based classifier:
   - operational, filesystem, code, process, experiment, and tool questions to Protocosmobot;
   - synthesis, critique, philosophy, theory, and long-form explanation to OmegaClaw.
6. Tie-break by current load, confidence, or deterministic precedence.

If the human explicitly requests both agents' independent views, the governor may issue two response leases, but the exception should be deliberate and visible.

6.3.2 Lease semantics

A response lease prevents two agents from independently answering the same event.

```
{
  "lease_id": "lease_01J...",
  "scope": {
    "event_id": "evt_01J...",
    "task_id": "a6-resweep",
    "state_version": 42
  },
  "holder": "protocosmo",
  "reviewers": ["omegaclaw"],
  "public_slots": 1,
  "expires_at": "2026-07-24T18:47:00Z",
  "status": "ACTIVE"
}
```

Leases should be short-lived and renewable. If the owner fails, the lease can expire and be reassigned. A completed lease records the outbound event identifier so retries do not create another answer.

6.4 Task and thread lifecycle

Threads should have explicit lifecycle state rather than relying on conversational implication.

Suggested states:

```
OPEN | WAITING_FOR_TOOL | WAITING_FOR_HUMAN | REVIEWING | ANSWERED | CLOSED |
REOPENED
```

A closed thread should not be reopened by an acknowledgment, repeated status, or sibling echo. Reopening requires one of:

- new evidence;
- a human follow-up;

- a consequential correction;
- a changed project state version;
- an explicit reopen command.

This rule directly prevents "the echo loop is resolved" from becoming the topic of a new echo loop.

6.5 Idempotency and duplicate detection

6.5.1 Exact idempotency

The gateway should record handled event IDs and source message IDs. A retry or reconnect must preserve a `retry_of_event_id` link. Exact duplicates are dropped without model use.

A useful key is:

```
(project_id, task_id, event_type, state_version, intended_recipient)
```

Only one public response should normally be emitted for this key.

6.5.2 Near-duplicate and novelty detection

Exact hashes are insufficient because agents often restate the same content with different wording. The governor should compute a semantic fingerprint and compare the proposed message with recent messages in the same task or thread.

A pragmatic novelty score can combine:

```
novelty = 0.45 * semantic_distance
        + 0.25 * new_fact_fraction
        + 0.20 * state_version_change
        + 0.10 * new_action_or_decision
```

The weights are tunable. The important feature is that a message with no new fact, no state change, and no new action should be suppressible even when its wording differs.

For early deployment, semantic duplicate detection should be advisory or shadow-only. Deterministic classes such as acknowledgments and visible `NO_REPLY` can be suppressed immediately.

6.6 Extended smart router

The existing smart-ish router should be extended from model choice to action choice.

Action	Meaning
`DROP`	Record the event and take no further action. No model call and no outbound message.
`LOG_ONLY`	Write operational state, incident count, or evidence to logs/project records without public output.
`AGGREGATE`	Add the event to a daily digest, incident summary, or batch response.
`TEMPLATE`	Produce a deterministic or parameterized message without an LLM, such as a concise approval request or first incident alert.
`CHEAP_MODEL`	Use a small or inexpensive model for classification, compression, or simple direct answers.

Action	Meaning
NORMAL_MODEL	Use the normal capable model for substantive operational or conceptual work.
DEEP_MODEL	Use a high-capability reasoning model for difficult technical analysis, research, theorem work, or explicitly requested depth.

The router should also attach a response contract rather than merely a token limit.

```
{
  "message_class": "bot_bot_critique",
  "soft_word_limit": 140,
  "hard_word_limit": 240,
  "required_sections": ["new_objection", "recommended_revision"],
  "forbidden_behaviors": [
    "acknowledgment_only",
    "full_status_recap",
    "new_project_offer"
  ],
  "exceed_soft_limit_when": [
    "safety_critical",
    "multiple_independent_consequences",
    "human_requested_depth"
  ]
}
```

6.7 Private bot-to-bot review bus

Operational coordination should not normally occur as public Telegram conversation. The gateway should provide typed internal messages:

```
REVIEW_REQUEST | EVIDENCE_REQUEST | TASK_HANDOFF | OBJECTION | DECISION_PROPOSAL
```

Example:

```
{
  "type": "REVIEW_REQUEST",
  "task_id": "petta-memory-compileadd-gate",
  "from": "protocosmo",
  "to": "omegaclaw",
  "question": "Identify the strongest flaw in the proposed gate.",
  "max_words": 120,
  "public": false,
  "deadline_seconds": 90
}
```

The reviewer returns a compact structured critique. The public owner integrates it into one answer. This retains cross-agent intelligence while eliminating parallel public essays.

6.8 Context construction and project-state snapshots

Each project should maintain a compact durable snapshot. A worker should not need to reread broad chat history on every invocation.

```
{
  "project_id": "omegasim",
  "state_version": 42,
  "current_goal": "Run stratified A6 resweep with CLA diagnostics",
  "last_material_result": "Tight region produced trivial grammars",
  "open_decisions": [],
  "active_blockers": [],
  "next_action": "Run next six unexplored cells",
  "evidence_refs": [
    "PROJECT.md#current-state",
    "runs/a6_stratum_04/report.json"
  ],
  "last_public_reported_version": 39
}
```

The context builder should retrieve:

- the current human message;
- the compact project snapshot;
- a bounded number of relevant evidence excerpts;
- the active response contract;
- optionally a private reviewer response.

It should not include repeated operational logs, old acknowledgments, or entire transcript histories unless specifically necessary.

6.9 Cron workers and update aggregation

Cron and project workers should write state, not directly publish routine updates.

Each worker result should declare:

```
{
  "project_id": "threadkeeper",
  "state_version": 103,
  "material_delta": true,
  "decision_required": false,
  "urgent": false,
  "summary": "Full-batch preflight now rejects malformed late calls atomically.",
  "evidence": ["commit:1ef286a", "tests:355"],
  "next_action": "Extend schema parity tests"
}
```

A single digest publisher runs once per day and includes only projects with meaningful state changes since their last public report. The default human-facing structure is:

1. decisions or approvals needed;
2. blockers and failures;

3. scientific or architectural conclusions that changed;
4. compact one-line deltas for other changed projects;
5. links or pointers to detailed evidence.

Immediate updates bypass the digest only for explicit exceptions:

- human approval required;
- safety, security, authority, or spending threshold reached;
- irrecoverable failure after retry policy;
- material change to scientific conclusion;
- direct human request for live updates;
- time-sensitive blocker where delay would waste substantial resources.

6.10 Incident manager

Repeated operational failures should be represented by one incident object.

```
{
  "incident_id": "inc_provider_503_20260724",
  "class": "MODEL_PROVIDER_UNAVAILABLE",
  "status": "OPEN",
  "first_seen": "2026-07-24T08:02:00Z",
  "last_seen": "2026-07-24T08:12:00Z",
  "occurrence_count": 9,
  "diagnosis": "upstream_503",
  "policy": "exponential_backoff",
  "last_public_notice_state": "OPEN_INITIAL",
  "next_public_notice_when": ["RECOVERED", "ESCALATED_30_MIN"]
}
```

The first occurrence may produce a concise alert. Later occurrences update counters silently. A second public message is sent only when diagnosis, severity, required human action, or recovery status changes.

6.11 Probe-before-prose gate

Before allowing a long response about an operational uncertainty, the governor or agent workflow should ask whether a cheap safe test exists. Examples include:

- file readability or existence;
- commit reachability;
- process or cron status;
- pod inventory;
- whether a message explicitly addressed the agent;
- whether a result artifact already exists;

- whether the project state version changed.

A configurable policy might be:

> Before generating more than 120 words about an operational uncertainty, perform the cheapest safe discriminating test or state briefly why such a test is unavailable.

This policy belongs partly in agent instructions and partly in workflow code. It should not block conceptual reasoning where no cheap empirical probe exists.

6.12 Egress governor

The egress governor is a final safety net. It should examine every candidate outbound message.

6.12.1 Hard suppression patterns

Messages that are exactly or effectively equivalent to the following should normally become no-send results:

```
NO_REPLY
Noted.
Acknowledged.
Nothing to add.
Staying quiet.
Already covered.
Not my task.
I will take a look.
Let me inspect.
Continuing...
```

Pattern matching should be supplemented with semantic classification, but deterministic patterns provide immediate value.

6.12.2 Candidate outbound schema

```
{
  "lease_id": "lease_01J...",
  "event_id": "evt_01J...",
  "agent_id": "protocosmo",
  "message_class": "direct_technical_answer",
  "text": "...",
  "claims": ["..."],
  "new_facts": ["..."],
  "requested_action": null,
  "state_version": 42,
  "references": ["artifact:report.json"],
  "send_intent": true
}
```

The egress governor verifies:

- the sender holds the active lease;
- the thread is not closed without a valid reopen condition;

- the output is not a near-duplicate of recent messages;
- the output contains the required response-contract fields;
- the output does not expose internal-only logs or secrets;
- multipart output is complete before delivery;
- length overruns are justified by policy.

7 Agent-Specific Integration

7.1 Protocosmobot as execution and evidence agent

Protocosmobot should remain the primary agent for:

- code, filesystem, repository, process, and infrastructure work;
- experiments and project workers;
- evidence gathering and verification;
- creating and modifying artifacts;
- operational status and approval requests.

Its tools and skills should return structured internal results by default. Public prose should be generated only after the governor classifies the event as human-visible.

Protocosmobot's local policy should include:

1. Perform ordinary tool sequences silently.
2. Do not narrate each command, retry, file lookup, or provisioning step.
3. Report completion, consequential failure, blocker, or approval request.
4. Store full commands, hashes, tests, and traces in project artifacts or logs.
5. When reporting progress, send only the delta since the last reported state version.
6. Do not interpret another bot's acknowledgment as a new task.
7. When a conceptual synthesis is needed, request a private OmegaClaw review rather than inviting a public second answer.

7.2 OmegaClaw as synthesis and reasoning agent

OmegaClaw should normally receive curated context rather than the raw activity stream. It is well suited for:

- conceptual synthesis;
- criticism and alternative interpretations;

- scientific and philosophical analysis;
- prioritization among options;
- converting structured evidence into a human-readable conclusion;
- long-form writing when requested.

Its local communication policy should include:

1. Do not paraphrase operational updates merely to show comprehension.
2. Do not acknowledge sibling messages unless acknowledgment changes shared state.
3. Do not answer a task owned by Protocosmobot unless assigned as reviewer or explicitly addressed.
4. Return compact private reviews when acting as reviewer.
5. Prefer a conclusion, strongest reasons, uncertainty, and recommendation over a broad essay.
6. Put durable deep treatments into artifacts and post a short pointer in chat.
7. Do not routinely end with an unsolicited proposal to create another formalization, paper, or project.

7.3 When both agents should contribute

Both agents should contribute when the task combines operational evidence and difficult interpretation. Examples include:

- interpreting an experiment result;
- deciding whether a security hardening change is sufficient;
- evaluating a proof-engine report;
- analyzing a failed training run and choosing the next experiment;
- preparing a strategic daily digest from many project workers.

The interaction should be private and bounded:

1. Protocosmobot gathers evidence and proposes a factual summary.
2. OmegaClaw receives a targeted review question with a short word budget.
3. Protocosmobot or the designated spokesperson integrates the critique.
4. One public response is sent.

The human can explicitly request independent answers when diversity itself is valuable.

8 Algorithms and Decision Procedures

8.1 Inbound event processing

```
def process_inbound(event):
    event = normalize(event)

    if exact_duplicate(event):
        return Decision.drop("EXACT_DUPLICATE")

    if is_own_message_loopback(event):
        return Decision.drop("OWN_MESSAGE_LOOPBACK")

    if incomplete_multipart(event):
        buffer(event)
        return Decision.defer("WAITING_FOR_PARTS")

    if update_existing_incident(event):
        if incident_requires_public_transition(event):
            return incident_notice_decision(event)
        return Decision.log_only("INCIDENT_REPEAT")

    if routine_project_update(event):
        if not event.material_delta:
            return Decision.log_only("NO_MATERIAL_DELTA")
        if not immediate_update_exception(event):
            aggregate_into_daily_digest(event)
            return Decision.aggregate("DAILY_DIGEST")

    owner = resolve_public_owner(event)

    if owner is None:
        return Decision.drop("NO_ELIGIBLE_RESPONDER")

    if active_lease_conflicts(event, owner):
        return Decision.drop("RESPONSE_ALREADY_CLAIMED")

    lease = issue_lease(event, owner)
    reviewers = choose_private_reviewers(event, owner)
    route = choose_router_action(event, owner)
    contract = choose_response_contract(event, route)
    context = build_compact_context(event, owner, contract)

    return Decision.admit(
        owner=owner,
        reviewers=reviewers,
        route=route,
        contract=contract,
        context=context,
        lease=lease,
    )
```

8.2 Public owner resolution

```
def resolve_public_owner(event):
    if event.addressing.explicit_agents:
        return highest_priority_explicit_agent(event)
```

```
    if event.addressing.reply_to_agent:
        return event.addressing.reply_to_agent

    if task_owner := lookup_task_owner(event.scope.task_id):
        return task_owner

    if spokesperson := channel_spokesperson(event.source.channel_id):
        return spokesperson

    scores = {
        "protocosmo": operational_fit(event),
        "omegaclaw": synthesis_fit(event),
    }
    return deterministic_argmax(scores)
```

8.3 Egress processing

```
def process_egress(candidate):
    if not valid_lease(candidate):
        return Suppress("NO_ACTIVE_LEASE")

    if no_send_semantics(candidate.text):
        complete_lease_without_send(candidate.lease_id)
        return Suppress("NO_REPLY_SEMANTICS")

    if contains_internal_tool_narration(candidate):
        candidate = remove_or_summarize_tool_narration(candidate)

    if semantic_duplicate_of_recent(candidate):
        complete_lease_without_send(candidate.lease_id)
        return Suppress("SEMANTIC_DUPLICATE")

    if not satisfies_contract(candidate):
        candidate = repair_with_small_model_or_template(candidate)

    if incomplete_multipart(candidate):
        buffer(candidate)
        return Suppress("WAITING_FOR_OUTPUT_PARTS")

    outbound_id = send(candidate)
    record_public_response(candidate, outbound_id)
    complete_lease(candidate.lease_id, outbound_id)
    return Sent(outbound_id)
```

8.4 Daily digest construction

```
def build_daily_digest(project_states):
    changed = [
        p for p in project_states
        if p.state_version > p.last_public_reported_version
        and p.material_delta
    ]

    decisions = [p for p in changed if p.decision_required]
```

```
    blockers = [p for p in changed if p.active_blockers]
    conclusions = [p for p in changed if p.conclusion_changed]
    routine = [
        p for p in changed
        if p not in decisions + blockers + conclusions
    ]

    return render_digest(
        decisions=decisions,
        blockers=blockers,
        conclusions=conclusions,
        routine=routine,
        max_words=900,
    )
```

9 Response Classes and Progressive Disclosure

The system should use response classes rather than one universal style.

Class	Default length	Expected content
Receipt or acknowledgment	0 words	Hidden receipt, state transition, or reaction. No LLM call where possible.
Routine project delta	60–100 words	Changed result, evidence pointer, next action, blocker if any.
Bot-to-bot proposal	100–160 words	One proposal, rationale, explicit question.
Bot-to-bot critique	100–160 words	Strongest objection, correction, and recommended revision.
Human decision request	80–160 words	Decision needed, options, recommendation, consequence of delay.
Direct substantive answer	200–400 words	Conclusion, strongest reasons, uncertainty, recommended next step.
Deep requested analysis	Flexible	Full treatment, preferably stored as a durable artifact with compact chat summary.
Operational incident	40–100 words	What failed, current impact, retry policy, whether human action is needed.

Progressive disclosure should be targeted. Instead of “would you like more detail?”, the response may expose named expansion handles:

```
DETAIL: statistical caveat
DETAIL: implementation architecture
EVIDENCE: test results
ALTERNATIVE: conservative design
```

A follow-up request can retrieve only the named detail. This reduces the tendency of broad clarification prompts to trigger another general essay.

10 Channel Policies

Every channel should have a policy object. The current Telegram rooms and future OmegaHive channels may differ substantially.

```
{
  "channel_id": "telegram:protobots",
  "channel_type": "mixed_human_ai",
  "visibility": "human_visible",
  "default_spokesperson": "protocosmo",
  "allowed_public_responders": 1,
  "bot_bot_public_discussion": "exception_only",
  "tool_narration": "forbidden",
  "routine_update_policy": "daily_digest",
  "incident_policy": "coalesced",
  "max_bot_turns_per_state_version": 2,
  "retention": "full_log",
  "privacy_label": "private_group"
}
```

Recommended channel types include:

Human-direct A human converses with one selected agent. Other agents participate only through private review.

Mixed human/AI project room
Humans and agents share a room. One project spokesperson answers by default.

AI-only coordination room
Structured task handoffs, review requests, and evidence exchange. Logged but not sent to social media.

AI-only deliberation room
A bounded committee explores alternatives and returns one synthesis artifact.

Public social channel Strongest egress policy, moderation, privacy filtering, and designated spokesperson.

Operational log stream
Machine-readable events; not treated as conversational input unless an incident transition occurs.

11 Failure Modes and Safety Considerations

11.1 False silence

The largest risk is suppressing a message that contained a useful warning or correction. Mitigations include:

- shadow-mode deployment;
- conservative deterministic suppression first;
- bypass for safety, security, authority, cost, and explicit human addressing;

- logging every suppression decision with reason codes;
- easy human command to inspect suppressed events;
- periodic audit samples reviewed by a strong model or human.

11.2 Over-centralization

A shared governor can become a bottleneck or single point of failure. The design should therefore:

- keep the governor policy-driven and relatively simple;
- preserve direct emergency paths for high-severity agents;
- use durable event logs so decisions can be replayed;
- allow local channel governors in a larger hive;
- avoid embedding substantive domain reasoning in the governor.

11.3 Lease deadlock or abandonment

An agent may acquire a lease and fail. Leases need timeouts, heartbeats only while real work is underway, and deterministic reassignment. A lease expiration should not itself generate public chatter.

11.4 Semantic deduplication errors

Two messages may be semantically similar but differ in an important caveat. Early semantic suppression should therefore operate in shadow mode or require strong confidence. New numerical evidence, changed state version, changed requested action, explicit correction, or high-severity flags should override semantic similarity.

11.5 Private review leakage

AI-only channels may contain speculative, sensitive, or low-confidence material. Egress must enforce privacy labels and ensure that public synthesis does not expose internal-only content without authorization.

11.6 Human confusion about who answered

The public response should clearly retain the identity of the designated agent. Internal reviewers can be credited when useful, but the system should not blur agent identity or create the appearance that one bot performed another bot's tools.

12 Observability and Evaluation

The rollout should be measured, not judged only by subjective quietness.

12.1 Core metrics

Metric	Interpretation
LLM calls per human-visible useful response	Direct measure of coordination overhead.

Metric	Interpretation
Tokens per material project delta	Measures whether routine work is communicated efficiently.
No-op invocation rate	Fraction of LLM calls that result in acknowledgment, silence marker, or no new content.
Duplicate outbound rate	Fraction of public messages semantically duplicating a recent message.
Parallel-response rate	Fraction of human events receiving more than one unrequested public agent answer.
Tool-narration rate	Fraction of public messages primarily describing intermediate execution steps.
Incident amplification factor	Public incident messages divided by distinct incident state transitions.
False-silence rate	Suppressed events later judged to have contained useful or necessary content.
Human clarification burden	Number of times the human must ask what is actually happening after agent updates.
Time to decision	Elapsed time from decision-relevant event to clear human decision request or answer.
Useful-information retention	Human or reviewer rating of whether important caveats and options remain available.

12.2 Audit records

Every governor decision should be recorded:

```
{
  "event_id": "evt_01J...",
  "decision": "DROP",
  "reason_codes": ["ACK_ONLY", "THREAD_CLOSED"],
  "policy_version": "ocg-0.3.1",
  "candidate_owner": "omegaclaw",
  "model_call_avoided": true,
  "estimated_tokens_avoided": 850,
  "review_sample_bucket": "weekly_1_percent"
}
```

This supports rollback, policy tuning, and scientific analysis of hive communication behavior.

13 Deployment and Migration Strategy

13.1 Phase 0: baseline instrumentation

Before changing behavior, instrument the current system for one to two weeks:

- assign stable event IDs;
- record origin chains;
- classify message types;

- measure model calls, tokens, duplicates, acknowledgments, and public responder counts;
- construct project-state versions without suppressing output.

13.2 Phase 1: shadow governor

The governor makes decisions but does not enforce them. Logs should show:

- which messages would have been dropped;
- which agent would have owned each event;
- which updates would have been aggregated;
- estimated tokens saved;
- potential false-silence cases.

13.3 Phase 2: deterministic suppression

Activate only very high-confidence rules:

- suppress exact `NO_REPLY`;
- suppress exact acknowledgments and "staying quiet" messages;
- block own-message loopback;
- coalesce exact repeated incidents;
- stop public tool-step narration from known worker event types;
- aggregate "no material change" project updates.

13.4 Phase 3: shared response leases

Enable one public owner per event. Begin with mixed human/AI project channels. Preserve an override for explicit human requests for multiple agents.

13.5 Phase 4: private review bus

Move routine bot-to-bot critiques and evidence requests off Telegram. Bound reviews by question, time, and response length.

13.6 Phase 5: daily digest and compact project state

Migrate cron workers to state-writing mode and deploy one digest publisher. Use compact project snapshots for context construction.

13.7 Phase 6: semantic novelty and adaptive budgets

After sufficient labeled data is collected, activate semantic near-duplicate suppression conservatively. Adapt word and model budgets based on message class, project risk, and human preferences.

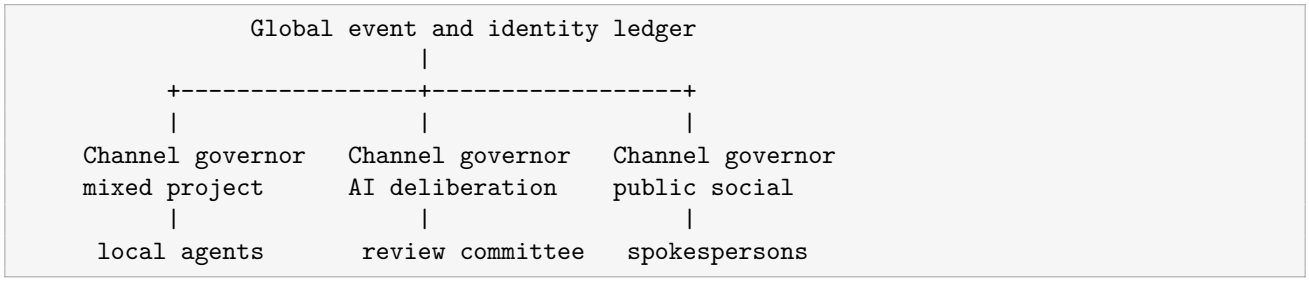
14 Extension to a Richer OmegaHive

14.1 From a two-agent governor to a communication fabric

A larger OmegaHive may contain many specialized agents, such as coding, mathematical, scientific, communications, governance, safety, memory, project-management, and social agents. It may also contain many channel types:

- AI-only task and review channels that are fully logged but not placed on Telegram, Slack, or another social platform;
- mixed human/AI project rooms resembling the current Telegram channels;
- human-only governance or decision rooms;
- public or semi-public social channels;
- machine-only operational event streams;
- temporary deliberation rooms created for one problem and closed afterward.

At this scale, a single central governor may become too coarse. The recommended architecture is federated.



Each channel governor enforces local policy. The global ledger provides event identity, agent identity, project state versions, privacy labels, task ownership, and cross-channel deduplication.

14.2 Typed channels and boundary brokers

Every channel should declare:

- participants and eligible agent roles;
- human visibility;
- privacy and retention class;
- default spokesperson;
- maximum public responders;
- whether bot-to-bot free discussion is allowed;
- response and token budgets;
- escalation paths;

- whether content may cross into another channel.

A **boundary broker** governs movement from internal AI-only channels into mixed or public channels. Internal deliberation may be broad and exploratory, but the boundary broker produces a concise, provenance-linked synthesis and enforces privacy constraints.

14.3 Topic and role subscriptions

Agents should not ingest every message. They subscribe by role, project, topic, and event type.

Example subscriptions:

```
{
  "agent_id": "mathmaxxer",
  "subscriptions": [
    {"project": "rh", "event_types": ["proof_target", "counterexample"]},
    {"role": "formal_reviewer", "severity": ["high"]}
  ],
  "exclude": ["routine_build_log", "acknowledgment", "social_banter"]
}
```

This prevents large-agent hives from becoming all-to-all broadcast networks.

14.4 Bounded deliberation committees

When diversity is useful, the governor creates a temporary committee with explicit roles:

- proposer;
- adversarial critic;
- evidence checker;
- synthesizer;
- optional safety or governance reviewer.

The committee operates in an AI-only logged room with a bounded number of turns. It outputs one decision artifact and one human-facing summary. This is preferable to allowing every agent to answer independently in a mixed channel.

14.5 Attention budgets

Each project, channel, and task can carry a communication budget:

```
{
  "task_id": "omegasim-a7-design",
  "max_agent_turns": 8,
  "max_deep_model_calls": 2,
  "max_total_output_tokens": 7000,
  "max_public_messages": 2,
  "deadline": "2026-07-26T18:00:00Z"
}
```

Agents may request a budget extension with a concrete justification. This makes thoughtful exceptions possible without allowing unbounded conversational drift.

14.6 Cross-channel state and duplicate prevention

The same project may appear in an AI-only research room, a mixed human project channel, and a public update channel. The global event ledger should ensure that one result is not independently summarized by many agents in every room.

A result event can have channel-specific publication states:

```
{
  "result_id": "res_omegasim_0042",
  "internal_research": "FULL_ARTIFACT_AVAILABLE",
  "mixed_project": "SUMMARY_PUBLISHED",
  "public_social": "NOT_ELIGIBLE",
  "daily_digest": "INCLUDED"
}
```

14.7 Reputation and confidence without chatter

A richer hive may use agent reputation or confidence to choose owners and reviewers. These values should affect routing silently rather than being continuously discussed in-channel. For example, a high-reliability coding agent may receive operational ownership while a high-quality critic is assigned privately.

14.8 Governance, moderation, and social behavior

Mixed and public channels need stronger policy than AI-only rooms:

- designated spokesagents;
- privacy redaction and secret scanning;
- rate limits and cooldowns;
- explicit distinction between agent speculation and project commitments;
- human override and correction paths;
- moderation of conflict escalation and repetitive bickering;
- provenance links for consequential claims.

The aim is not to remove personality or spontaneous social dynamics. It is to ensure that such dynamics are intentional features of a social room rather than accidental consequences of system-event routing.

15 Recommended Initial Configuration for the Current Proto-Hive

A practical starting configuration is:

Setting	Recommended value
Default public responder	Explicitly addressed agent; otherwise project owner; otherwise Protocosmobot for operational content and OmegaClaw for conceptual content.
Maximum public responders	1, unless the human explicitly requests independent views.
Private reviewer	Optional one-agent review for difficult or consequential tasks.
Routine project updates	Once daily, only for material deltas.
Tool narration	Internal only.
Visible `NO_REPLY`	Hard-suppressed.
Acknowledgments	Hidden receipts or reactions; no LLM call when possible.
Repeated incidents	One open notice, silent occurrence count, one recovery or escalation notice.
Bot-to-bot public turns	Maximum two substantive turns per state version.
Long analysis	Chat summary plus durable artifact.
Operational uncertainty	Cheap probe before response above 120 words.
Semantic duplicate suppression	Shadow mode initially; active only after audit.

16 Conclusion

The present proto-hive does not need less intelligence. It needs a communication substrate that makes intelligence selective about when to speak, who should speak, and what information belongs in public conversation.

The highest-value change is to move response admission, ownership, idempotency, update aggregation, and no-op suppression into a shared gateway-level governor. Agent prompts and length limits remain useful, but they become the final layer of refinement rather than the sole mechanism of control.

The architecture preserves the useful asymmetry between Protocosmobot and OmegaClaw: one gathers evidence and acts; the other synthesizes and critiques. Their collaboration becomes private, targeted, and bounded, while humans normally receive one coherent answer. Routine project activity becomes durable state plus one daily digest. Errors become incidents rather than repeated conversational events. Silence becomes a real action. Deep reasoning remains available when it is genuinely useful.

The same design scales naturally to a larger OmegaHive by federating channel governors around a shared identity and event ledger, separating internal AI deliberation from human-visible output, and using explicit spokesagents, response leases, attention budgets, and boundary brokers.

A Detailed Implementation Plan for an OpenClaw Coding Agent

This appendix is written as an actionable plan for an OpenClaw coding agent. The agent should execute it incrementally, preserve current functionality, and provide evidence for every gate.

A.1 Implementation mandate

Implement a shared OmegaHive Conversation Governor around the existing OpenClaw gateway used by Protocosmobot and OmegaClaw. The implementation must reduce duplicate and no-op model calls while preserving explicit human addressing, safety-critical intervention, deep requested analysis, and current tool functionality.

Do not begin by rewriting the agents. Begin by discovering the actual event flow and inserting observability. Use small commits, provider-free tests where possible, and a shadow-mode rollout.

A.2 Operating constraints for the implementing agent

1. Do not modify live routing until a provider-free replay harness exists.
2. Do not suppress messages in production until shadow logs have been reviewed.
3. Do not publish raw user transcripts or private artifacts to public repositories.
4. Preserve current model-router behavior behind a feature flag.
5. Keep policy configuration external to core code where practical.
6. Use deterministic checks before model-based classification.
7. Record exact commands, tests, commits, and rollback instructions.
8. Avoid narrating every implementation step to the human; report milestone completion, blockers, and decisions only.

A.3 Phase A: repository and runtime discovery

A.3.1 Tasks

1. Identify the repository or repositories containing:
   - OpenClaw gateway ingress;
   - Telegram adapter;
   - agent dispatch;
   - existing smart router;
   - model invocation wrappers;
   - cron/project worker publication paths;
   - OmegaClaw gateway client;
   - message persistence or thread state.

2. Produce an event-flow map from inbound Telegram update to outbound send.
3. Identify every place where an LLM call can occur before a public response.
4. Identify whether both agents share one process, one host, one queue, or only an inference endpoint.
5. Identify current message IDs, reply metadata, bot sender IDs, channel IDs, and any existing idempotency keys.
6. Locate current configuration and feature-flag mechanisms.

A.3.2 Deliverable

Create `docs/conversation_governor/current_event_flow.md` containing:

- actual module and function names;
- an ASCII flow diagram;
- proposed insertion points for ingress and egress governors;
- risks and unknowns;
- confirmation of whether a shared pre-inference boundary exists.

A.3.3 Acceptance criteria

- At least one real inbound event is traced end to end using logs or tests.
- The exact location where model selection occurs is identified.
- The exact location where Telegram sends occur is identified.
- No live behavior is changed.

A.4 Phase B: event schema and durable ledger

A.4.1 Tasks

1. Implement a canonical `EventEnvelope` data structure.
2. Add stable event IDs and preserve source message IDs.
3. Add origin chain, retry lineage, sender kind, explicit mentions, reply-to agent, channel type, project ID, task ID, thread ID, and state version fields.
4. Implement a lightweight durable store, initially SQLite or the project's existing database.
5. Add tables or collections for:
   - events;
   - governor decisions;
   - response leases;

- thread state;
- project report state;
- incidents;
- recent semantic fingerprints.

6. Ensure that storage failures fail open for conversational delivery during early rollout, while logging the failure.

A.4.2 Suggested module layout

Adapt names to the repository, but aim for a separation like:

```
conversation_governor/
    __init__.py
    models.py
    event_normalizer.py
    ledger.py
    admission.py
    ownership.py
    leases.py
    incidents.py
    digests.py
    context_builder.py
    egress.py
    policies.py
    semantic_dedup.py
    metrics.py
    feature_flags.py
```

A.4.3 Acceptance criteria

- Event envelopes round-trip through the store deterministically.
- Retried source messages can be linked to their original events.
- Bot-originated outbound messages retain origin metadata when re-ingested.
- Provider-free unit tests cover malformed envelopes and storage recovery.

A.5 Phase C: transcript-derived replay corpus

A.5.1 Tasks

Construct a private test corpus from the supplied transcript patterns. Do not publish the raw conversations. Convert them into minimal synthetic cases representing:

1. acknowledgment loop;
2. visible `NO_REPLY`;
3. duplicate extended analysis;

4. repeated 503 traceback;
5. tool-step narration during provisioning;
6. two agents answering one human message;
7. message addressed to one agent but answered by another;
8. multipart message misinterpreted before completion;
9. “no material change” project update;
10. file-access speculation resolved by a cheap probe;
11. explicit request for both agents’ independent opinions;
12. high-severity correction from a non-owner agent.

Represent each case as input events plus expected governor decisions.

A.5.2 Example fixture

```
{
  "name": "ack_loop_suppression",
  "events": [
    {"sender": "protocosmo", "text": "Work complete. NO_REPLY"},
    {"sender": "omegaclaw", "text": "Noted that work is complete."}
  ],
  "expected": [
    {"decision": "DROP", "reason": "NO_REPLY_SEMANTICS"},
    {"decision": "DROP", "reason": "ACK_ONLY"}
  ]
}
```

A.5.3 Acceptance criteria

- At least twelve replay scenarios exist.
- They run without provider calls.
- Each scenario asserts both the decision and the absence or presence of outbound sends.
- The replay runner can compare legacy behavior with shadow governor behavior.

A.6 Phase D: shadow admission engine

A.6.1 Tasks

1. Implement deterministic rules for exact duplicates, own-message loopback, explicit addressing, repeated incidents, routine unchanged updates, and visible no-reply phrases.
2. Implement owner resolution without enforcing it.
3. Add decision reason codes.

4. Invoke the shadow governor on every inbound event while leaving legacy dispatch unchanged.
5. Record estimated model calls and tokens that would have been avoided.
6. Add a daily internal shadow report.

A.6.2 Acceptance criteria

- Shadow mode cannot suppress or reroute production messages.
- Every decision is auditable by event ID.
- Replay corpus passes.
- A one-day live shadow report identifies candidate no-op and duplicate traffic.

A.7 Phase E: hard no-op suppression

Activate only deterministic egress suppression behind a feature flag.

A.7.1 Initial suppression classes

- exact `NO_REPLY`;
- exact or near-exact acknowledgments;
- "staying quiet," "nothing to add," and "already covered" when no new claim or action is present;
- own outbound message loopback;
- exact repeated system-event payloads;
- known intermediate tool-progress event types.

A.7.2 Required safeguards

- Never suppress explicit human messages.
- Never suppress safety, security, cost, authority, or irreversible-action alerts.
- Log candidate text and reason privately.
- Provide a command to inspect recent suppressed messages.
- Provide a single feature flag to disable all active suppression.

A.7.3 Acceptance criteria

- Replay corpus shows no regression in explicit human answers.
- Live canary produces no known false silence for at least forty-eight hours.
- Token and message counts decline measurably.

A.8 Phase F: response leases and ownership

A.8.1 Tasks

1. Implement lease creation, renewal, completion, expiration, and reassignment.
2. Integrate explicit mentions, reply-to metadata, project ownership, and role fit.
3. Require a valid lease before public egress from either agent.
4. Allow a policy exception for explicit human requests for multiple independent answers.
5. Add a conflict log when two agents attempt to acquire the same lease.

A.8.2 Acceptance criteria

- One human event produces one public answer in default mode.
- Explicit dual-agent requests produce two answers or one labeled comparative synthesis, according to policy.
- Lease expiration recovers from a crashed owner without duplicate sends.
- Retries after completed leases do not resend.

A.9 Phase G: private review bus

A.9.1 Tasks

1. Implement typed private review requests through the gateway.
2. Add bounded reviewer prompts and structured review returns.
3. Ensure private reviews are not automatically posted to Telegram.
4. Permit the public owner to include or ignore reviewer feedback.
5. Record reviewer provenance in internal logs and optionally in public artifact metadata.

A.9.2 Acceptance criteria

- A test task uses Protocosmobot as owner and OmegaClaw as private reviewer.
- Only one public response is emitted.
- The public response reflects at least one reviewer correction in the test fixture.
- Reviewer timeout does not block the owner indefinitely.

A.10 Phase H: project snapshots and digest migration

A.10.1 Tasks

1. Define `ProjectStateSnapshot` and state-version semantics.
2. Modify one low-risk cron worker to write state instead of sending Telegram updates.

3. Implement a daily digest generator.
4. Add immediate-update exception flags.
5. Migrate workers gradually, one project at a time.
6. Track `last_public_reported_version` per project.

A.10.2 Acceptance criteria

- The pilot project emits at most one routine daily update.
- Multiple same-day commits are summarized as one delta.
- Approval-required events still notify immediately.
- "No material change" creates no public message.

A.11 Phase I: incident coalescing

A.11.1 Tasks

1. Define incident classes and state transitions.
2. Convert provider failures, cron failures, and repeated tool errors into incident updates.
3. Send first-open, escalation, required-action, and recovery notices only.
4. Store raw traces in logs with secure access.

A.11.2 Acceptance criteria

- Ten repeated identical 503 events produce one initial notice and one recovery notice.
- A changed diagnosis produces a new notice.
- Raw trace remains retrievable for debugging.

A.12 Phase J: semantic deduplication

A.12.1 Tasks

1. Add semantic fingerprints using an existing embedding service or inexpensive local model.
2. Compare candidates only within bounded recent task/thread windows.
3. Extract simple structured novelty indicators: new numbers, new evidence refs, changed state version, changed requested action, correction markers.
4. Run in shadow mode and label false-positive cases.
5. Activate only above a high confidence threshold.

A.12.2 Acceptance criteria

- Exact paraphrases are reliably detected.
- New caveats or changed results are not suppressed in the evaluation set.
- High-severity and explicit-human-follow-up messages bypass semantic suppression.

A.13 Phase K: metrics dashboard and policy tuning

Implement counters and reports for:

- model calls avoided;
- estimated input and output tokens avoided;
- suppressed no-op messages;
- duplicate public responses prevented;
- incident repeats coalesced;
- daily digest compression ratio;
- false-silence reports;
- average time to useful answer;
- response ownership distribution between agents.

Do not optimize only for token reduction. Establish a utility audit that samples suppressed and compressed events.

A.14 Feature flags

At minimum, provide:

```
OCG_SHADOW_MODE
OCG_ACTIVE_EGRESS_SUPPRESSION
OCG_RESPONSE_LEASES
OCG_PRIVATE_REVIEW_BUS
OCG_DAILY_DIGEST
OCG_INCIDENT_COALESCING
OCG_SEMANTIC_DEDUP
OCG_COMPACT_CONTEXT
```

Feature flags should be independently reversible.

A.15 Testing strategy

A.15.1 Unit tests

Test schemas, reason codes, exact deduplication, mention parsing, lease transitions, thread reopening, incident transitions, and egress suppression.

A.15.2 Property tests

Useful properties include:

- one active public lease per default event scope;
- completed lease plus retry never yields a second send;
- explicit human addressing is never dropped by acknowledgment rules;
- state-version increase prevents stale-update suppression;
- incident occurrence count is monotonic;
- private review events cannot cross a public boundary without synthesis.

A.15.3 Replay tests

Replay synthetic transcript-derived sequences under legacy and governor modes. Assert outbound message counts and owners.

A.15.4 Canary tests

Use one low-risk private channel before mixed human channels. Preserve full logs and provide immediate rollback.

A.16 Suggested commit sequence

A clean incremental sequence might be:

1. Documentation and event-flow map.
2. Event schema and ledger, no behavior change.
3. Replay harness and synthetic fixtures.
4. Shadow admission decisions.
5. Deterministic egress no-op suppression.
6. Response leases.
7. Private review bus.
8. Project-state snapshot pilot.
9. Daily digest pilot.
10. Incident manager.
11. Semantic dedup shadow mode.
12. Metrics and rollout report.

Each commit should include tests and a concise migration note.

A.17 Definition of done for the first production release

The first production release is complete when:

1. Both Protocosmobot and OmegaClaw pass through one shared governor boundary.
2. Exact no-op and acknowledgment messages do not reach Telegram.
3. A human message receives one public answer by default.
4. Explicit requests for multiple agent views still work.
5. Repeated provider errors are coalesced.
6. At least one project worker reports through a daily digest instead of direct routine updates.
7. A private review request can influence one public synthesis.
8. All active behavior is feature-flagged and reversible.
9. Metrics show meaningful reduction in calls and messages without observed loss of important information.
10. Documentation describes configuration, troubleshooting, and rollback.

A.18 Implementation-agent reporting format

The OpenClaw implementation agent should report milestones using:

```
Completed:
Evidence:
Behavioral change:
Tests:
Risks or blocker:
Next gate:
```

It should not send a message for every command, file inspection, or minor edit.

B Example Policy Configuration

```
{
  "version": "ocg-policy-0.1",
  "global": {
    "default_public_slots": 1,
    "suppress_visible_no_reply": true,
    "suppress_ack_only": true,
    "tool_narration_public": false,
    "semantic_dedup_mode": "shadow",
    "explicit_human_addressing_bypass": true
  },
  "agents": {
    "protocosmo": {
      "roles": ["execution", "evidence", "operations", "project_owner"],
      "default_message_classes": ["status_delta", "decision_request"]
    },
```

```
    "omegaclaw": {
      "roles": ["synthesis", "critique", "theory", "long_form"],
      "default_message_classes": ["substantive_answer", "private_review"]
    }
  },
  "channels": {
    "telegram:protobots": {
      "type": "mixed_human_ai",
      "default_spokesperson": "protocosmo",
      "routine_updates": "daily_digest",
      "max_public_responders": 1,
      "max_bot_turns_per_state_version": 2
    },
    "internal:reviews": {
      "type": "ai_only_structured",
      "human_visible": false,
      "max_turns": 3,
      "retain_full_log": true
    }
  }
}
```

C Test Matrix

Scenario	Expected governor behavior	Critical assertion
Human addresses Protocosmobot	Admit Protocosmobot; optional private OmegaClaw review	Human receives one public answer.
Human addresses OmegaClaw	Admit OmegaClaw	Protocosmobot does not paraphrase the answer.
Human asks both agents separately	Issue two leases or comparative-synthesis policy	Explicit diversity request is preserved.
Bot posts `NO_REPLY`	Suppress at egress	Zero Telegram sends.
Sibling posts "noted"	Drop before inference when detectable	No LLM call.
Same 503 repeats ten times	Update one incident	At most one open notice before recovery/escalation.
Tool emits five progress steps	Log internally	Only completion or blocker becomes public.
Project commit count increases twice	Aggregate state	One daily delta.
Project has no material change	Log only	No public update.
Owner crashes after lease	Expire and reassign	At most one eventual public answer.
Message arrives in three parts	Buffer until complete	No response to partial content.
Near-duplicate answer with new number	Do not suppress	New result reaches human.

Scenario	Expected governor behavior	Critical assertion
Near-duplicate answer with no new content	Suppress or shadow-flag	Duplicate count decreases.
Non-owner detects security issue	Override lease policy	High-severity warning reaches owner/human.
AI-only review contains private speculation	Keep internal	Public synthesis does not leak private content.

D Future Research Directions

Once the practical governor is stable, the event log enables deeper study of multi-agent communication:

- learning policies that optimize useful information per token rather than brevity alone;
- estimating marginal value of an additional reviewer;
- detecting conversational attractors such as acknowledgment loops and status spirals;
- attention allocation as a market or ECAN-like process;
- causal attribution of which agent contribution improved the final answer;
- adaptive committee formation based on task novelty and risk;
- formal safety properties for one-response leases and privacy boundaries;
- modeling the governor as an OmegaHive social-regulation layer rather than a mere cost-control mechanism.

The most important research stance is that low-noise communication should be evaluated as a form of functional coordination, not cosmetic concision. The desired system speaks less because it routes attention and responsibility better, not because its agents have less to contribute.